COMPX523-26A Assignment 2 Report

Ensemble Classifier for Data Streams

Author: Necmi Kural

---

1. Design Overview

The assignment is split into a programming task and a tournament champion. Both share the same scaffolding (Online Bagging diversity, ADWIN per-member drift detection, and Leveraged Bagging for class imbalance), but they differ in the base learner:

Submission	File	Base learner	Drift detector	Default n
Programming (§2)	`ensemble_classifier.py`	Incremental Gaussian NB	CapyMOA ADWIN	10
Tournament (§3)	`champion_classifier.py`	Hoeffding Tree (capped 2 KB)	CapyMOA ADWIN (no-history subclass)	100

`EnsembleClassifier` is the literal implementation of the spec (GNB + ADWIN + Online Bagging + Leveraged Bagging). The champion is allowed to swap the base learner per PDF §3.1 and does so for accuracy reasons explained in §2.5.

---

2. Component Design Justifications

2.1 Online Bagging (Diversity Generation, PDF §2.1)

For each incoming instance and each base learner, draw $k \sim \text{Poisson}(\lambda)$ and re-train the learner on the instance k times. λ defaults to 1 (the canonical Online Bagging weight). The Champion draws all n Poisson samples in a single vectorised call (`numpy.random.poisson(λ, size=n)`) to keep the per-instance overhead constant.

2.2 Gaussian Naive Bayes Base Learner (PDF §2.2)

The programming-task ensemble uses an incremental GNB built on Welford's online algorithm. Sufficient statistics (count, running mean, sum of squared deviations) are stored per (class, feature) cell, so each learner costs $O(C \cdot F)$ numbers regardless of how many instances pass through. Welford's update is numerically stable across the 100 K-instance horizon, which matters because the naïve sum-of-squares formulation accumulates catastrophic cancellation error over long streams.

2.3 ADWIN Drift Detection (PDF §2.3)

Each ensemble member is monitored by an independent ADWIN instance (`capymoa.drift.detectors.ADWIN`, MOA-backed). The per-instance error signal (`0` if the learner predicted the true label, 1 otherwise) is pushed into ADWIN with `add_element`, and `detected_change` is queried immediately afterwards. When ADWIN reports a change we reset both the base learner and the detector (`reset(clean_history=True)`), letting that member re-learn the post-drift concept from scratch.

Memory hygiene. CapyMOA's `MOADriftDetector` base class appends every observation to a Python-side `data` list for user inspection. The actual change-detection algorithm runs on the Java side and never reads this list, but it grows unbounded otherwise: with 100 members across 37 K instances it inflated the champion's measured memory to 512 KB, breaching the 250 KB tournament cap. The champion therefore uses an `_ADWINNoHistory` subclass that overrides `add_element` to skip the bookkeeping append entirely. Memory becomes constant in stream length: `run_evaluation.py` measures 156.3 KB at the end of the 37 K sample stream and 156.1 KB at the end of the 100 K synthetic stream, so the value no longer depends on where in the stream the measurement lands. The fix preserves the full ADWIN change-detection semantics while eliminating the leak. The programming-task ensemble keeps the stock detector plus a 1 K-instance periodic purge, which is sufficient at n = 10. This matches the instructor's lecture warning that *"using the default drift detector as is will cause you to run out of memory"* and addresses it directly.

2.4 Leveraged Bagging for Class Imbalance (PDF §2.4)

A fixed-size buffer of recent labels tracks the live class distribution: a Python sliding list (size 500) for the programming ensemble, and a numpy ring buffer (size 2,000) for the champion. When an arriving instance belongs to a class whose share of that window is below the "minority" threshold (40 % for the programming ensemble, 35 % for the champion), the Poisson λ is scaled up proportionally to the imbalance ratio. The programming ensemble uses `λ_leverage = 6` and caps λ at 12; the champion uses `λ_leverage = 2` and caps λ at 8. The tighter champion cap was found empirically: an initial λ_leverage = 6 in the champion over-corrected on the sample CSV (minority recall 93 %, majority recall 17 %, TScore 0.44), while reducing to 2 produced the balanced per-class recalls (0.82 / 0.95 on sample, 0.96 / 0.97 on synthetic) that this submission ships.

2.5 Hoeffding Tree Champion (PDF §3.1)

The PDF explicitly permits replacing the base learner if it stays under 250 KB: *"You are not strictly limited to GNB if you find a better alternative."* GNB hit an accuracy ceiling on the sample data because its feature-independence and Gaussian-likelihood assumptions are both violated by the RBF data generator's mixture-of-Gaussians decision surfaces. Swapping in Hoeffding Trees with a tight `max_byte_size` cap lifted TScore from 0.61 (GNB) to 0.88 on the full sample stream and 0.96 on a synthetic 100 K-instance RBF stream with 4 drifts:

Base learner	Members	Per-tree cap	Sample TScore (37 K)	Synthetic TScore (100 K)
Vectorised GNB	100	n/a	0.606	n/a
HoeffdingTree	50	2 KB	0.82 (15 K subset)	n/a
HoeffdingTree	**100**	**2 KB**	**0.883**	**0.964**

`n = 100, max_byte_size = 2` KB was chosen as the default. Each tree is seeded differently so structural diversity supplements the Poisson sampling. Memory stays flat at ~156 KB regardless of stream length thanks to the no-history ADWIN subclass.

---

3. Memory Analysis

Memory budget (250 KB limit, PDF §3.2):

Component	Champion (HT, n=100)	Programming (GNB, n=10)
Base learners (Hoeffding trees / GNB stats)	72.8 KB	14.3 KB
ADWIN detectors (no-history subclass / purged stock)	79.5 KB	42.2 KB
Class ring buffer / history	2 KB (`int8 × 2000`)	4.2 KB (`list × 500`)
Schema + Classifier overhead	~2 KB	~3 KB
Measured total (`pympler.asizeof`)	**156 KB**	**64 KB**

Headroom on the champion: **94 KB** below the 250 KB limit. Memory is constant across the stream: `run_evaluation.py` measured 156.3 KB at the end of the 37 K sample stream and 156.1 KB at the end of the 100 K synthetic stream, confirming the value is independent of the instance count's relationship to any purge schedule.

Key savings: (i) `max_byte_size = 2` KB cap on each Hoeffding Tree, enforced by MOA's built-in memory manager (`stop_mem_management=True`); (ii) the `_ADWINNoHistory` subclass that overrides `add_element` to skip the unbounded Python-side history list; (iii) ring buffer vs. list for class history.

---

4. Parameter Choices

Parameter	Programming	Champion (tuned)
Ensemble size n	10	100
Base learner	Incremental GNB	Hoeffding Tree (`max_byte_size=2` KB)
λ_base (majority)	1	1
λ_leverage (minority multiplier)	6	2
Max λ for minority (hard cap)	12	8
Minority threshold	40 %	35 %
ADWIN δ	0.002	0.002 (no-history subclass)
Class window size	500	2,000
HT grace period	n/a	50
HT split confidence	n/a	0.05
Voting	Majority	Majority

---

5. Benchmark Results

End-to-end results from `run_evaluation.py` (the script committed alongside the source).

Sample test data: full 37,440 instances of `sample_test_csv.csv` (class imbalance 3.6 : 1)

Classifier	Accuracy	Balanced Acc	Macro F1	Tournament Score	Memory
NaiveBayes (baseline)	0.219	0.500	0.181	0.340	1.8 KB
`EnsembleClassifier` (n=10)	0.466	0.622	0.463	0.542	63.6 KB
`ChampionClassifier` (HT, n=100)	**0.920**	**0.883**	**0.883**	**0.883**	**156.3 KB**

Synthetic RBF stream: 100,000 instances, 4 drifts (1 abrupt + 3 gradual)

Classifier	Accuracy	Balanced Acc	Macro F1	Tournament Score	Memory
NaiveBayes (baseline)	0.623	0.612	0.606	0.609	1.8 KB
`EnsembleClassifier` (n=10)	0.842	0.840	0.841	0.841	—
`ChampionClassifier` (HT, n=100)	**0.964**	**0.964**	**0.964**	**0.964**	**156.1 KB**

The Champion's per-class recalls (0.82/0.95 on sample; 0.96/0.97 on synthetic) confirm the Leveraged Bagging mechanism stays balanced rather than over-correcting. The baseline NaiveBayes collapses to a near degenerate predictor on the imbalanced sample stream, which is exactly the failure mode the assignment is designed to expose. The Champion's memory footprint is essentially identical across the two streams (156.3 vs. 156.1 KB) and well below the 250 KB cap, confirming that the `_ADWINNoHistory` subclass eliminates the previous unbounded-growth leak.

Hyperparameter sweep methodology

A grid sweep on the sample CSV (8 K-instance subsets, 15 K confirmation runs, then a 37 K full-stream confirmation) explored `n_members` ∈ {10, 25, 50, 75, 100} and `max_byte_size` ∈ {1, 2, 4, 8} KB. Per-config metrics are tabulated in §2.5. n = 100 was selected over n = 50 after observing the TScore lift from 0.82 (n = 50, 15 K subset) to 0.88 (n = 100, full 37 K stream) on the sample data, with the no-history ADWIN keeping n = 100 well within the 250 KB cap. The synthetic-stream confirmation run (n = 100, full 100 K) then produced TScore 0.96.

---

6. Post-Mortem: Memory Enforcement Bug (added after grading)

The submitted version of this report claimed memory of 156 KB based on `pympler.asizeof`. The instructor's evaluator, which measures the JVM heap, reported ~25 MB instead and applied the memory penalty. Two bugs combined to produce that gap; both are fixed in the portfolio revision of `champion_classifier.py`.

Bug 1 — `pympler.asizeof` is the wrong tool for MOA wrappers. CapyMOA's `HoeffdingTree` is a thin Python wrapper over a Java object reached through JPype. `pympler.asizeof` walks the Python object graph and reports the bytes of the wrapper, not the JVM heap that holds the actual split statistics, node arrays, and attribute observers. Switching to MOA's own `measureByteSize()` (called via JPype on each `moa_learner`) gives the number the instructor's evaluator sees, and is what the per-tree `max_byte_size` cap is enforced against.

Bug 2 — `memory_estimate_period` defaults to 1,000,000. MOA only checks each tree's actual size against `max_byte_size` every `memory_estimate_period` instances. CapyMOA's default value is 1 M, so on the assignment-scale streams (37 K sample, 100 K synthetic) the check never fires. Trees grow freely to whatever the data triggers, and the cap is effectively ignored. Setting `memory_estimate_period=1000` makes enforcement effectively continuous without measurable overhead.

Fix. The portfolio revision:

1. Passes `memory_estimate_period=1000` when constructing each `HoeffdingTree`, so `max_byte_size` actually constrains tree growth.
2. Adds a `ChampionClassifier.measure_java_byte_size()` helper that sums `measureByteSize()` across base learners and is reported alongside the `pympler` number by `run_evaluation.py`.

3. Resizes the defaults to fit the realistic per-tree budget: `n_members=25, max_byte_size=8` KB (instead of `n=100, 2` KB), which keeps the *Java* footprint under 250 KB. The instructor's own `HoeffdingAdaptiveTree(n=10)` reference reached 0.88 TScore at 112 KB, validating the regime.

The §5 benchmark numbers were measured with the buggy configuration and are kept here for honesty; portfolio-quality re-runs of `run_evaluation.py` will produce different (and lower-but-valid) accuracy alongside the corrected memory readout.